

that we perform lattice diffusion on the subspace of *symmetric* lattice matrices. However, the lattice scores from $\hat{s}_{L,\theta}^l(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t)$ are generally not symmetric matrices. We address this issue with the following modification:

$$\begin{aligned} s_{L,\theta}^l(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) &= \tilde{s}_{L,\theta}^l(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) \mathbf{L}_t^\top \\ &= \mathbf{L}_t \mathbf{D} \tilde{\Phi}^l \mathbf{D}^\top \mathbf{L}_t^\top = \tilde{\mathbf{D}} \tilde{\Phi}^l \tilde{\mathbf{D}}^\top, \end{aligned} \quad (\text{A35})$$

where $\tilde{\Phi}^l = \text{diag}\left(\frac{\phi^l(\mathbf{m}_{ijk}^l)}{|\mathcal{E}| \cdot d_{ijk}^2}\right)$. Finally, the predicted layer-wise lattice scores are summed to obtain the predicted lattice score:

$$\mathbf{s}_{L,\theta}(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) = \sum_{l=1}^L \mathbf{s}_{L,\theta}^l(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t), \quad (\text{A36})$$

which is scale-invariant and equivariant under rotation. The equivariance derives from the way it is composed with the Cartesian coordinate matrix, and the scale invariance is due to the normalization happening inside $\tilde{\Phi}^l$. In particular, the diagonal entries of $\tilde{\Phi}^l$ related to the edges are normalized three times: they are divided by the total number of edges, and then multiplied twice by the inverse of the norm of the edge vectors. Given these properties, $\hat{\mathbf{s}}_\theta$ behaves like a symmetric stress tensor $\boldsymbol{\sigma}$, since the stress tensor is scale-invariant and equivariant under the rotation operator $\mathbf{R}$:

$$\boldsymbol{\sigma}'(\lambda \mathbf{M}) = \boldsymbol{\sigma}(\mathbf{M}), \quad (\text{A37})$$

$$\boldsymbol{\sigma}'(\mathbf{R} \mathbf{M}) = \mathbf{R} \boldsymbol{\sigma}(\mathbf{R} \mathbf{M}) \mathbf{R}^\top, \quad (\text{A38})$$

where we use λ to indicate the supercell replication operation.

A.8.2 Augmenting the input with lattice information

The chain-rule-based lattice score predictions from Eq. (A36) have shown to lack expressiveness for modeling the score of our Gaussian forward diffusion in our early experiments. We hypothesize that this is because our periodic GNN model is invariant to the particular choice of unit cell. For instance, it cannot distinguish the two structures in Fig. A1. To address this, we drop the invariance of the GNN w.r.t. equivalent choices of the unit cell by injecting information about the lattice angles into the internal representations. This means that the generative distribution is no longer invariant to the concrete choice of unit cell. We nonetheless note that any lattice can be *uniquely* transformed into its so-called Niggli-reduced cell [71]. We apply this transformation to all training data points and, consequently, side-step the loss of cell choice equivariance we introduce. Concretely, we concatenate the roto-translation invariant input edge representations $\mathbf{m}^{\text{inp}}$ with the cosines of the angles of the edge vectors w.r.t. the lattice cell vectors:

$$\hat{\mathbf{m}}_{ijk}^{\text{inp}} = \left(\mathbf{m}_{ijk}^{\text{inp}}, \cos(\mathbf{d}_{ijk}, \mathbf{l}^1), \cos(\mathbf{d}_{ijk}, \mathbf{l}^2), \cos(\mathbf{d}_{ijk}, \mathbf{l}^3) \right). \quad (\text{A39})$$

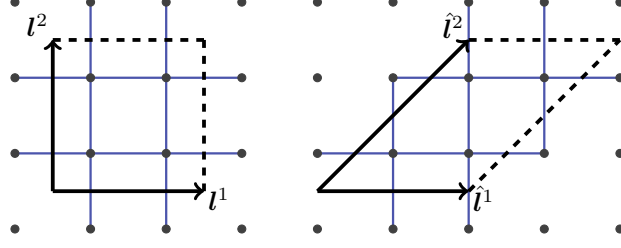


Fig. A1: Diagram showing two equivalent lattice choices $\mathbf{L}$ and $\hat{\mathbf{L}} = \mathbf{L}\mathbf{C}$ that lead to the same periodic structure. The dots represent atoms in the 2D periodic structures, and the blue lines indicate edges of atoms inside the unit cell to their four nearest neighbors, inside and outside the unit cell. Note that both choices of unit cell lead to indistinguishable structures, as indicated by the identical placement of atoms and equivalent edges. Here, $\mathbf{C} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$.

This additional information allows the model to distinguish the two cases in Fig. A1, while the internal representations remain invariant to rotation and translation.

A.9 Training loss

Our model is trained to minimize the following loss, which is a sum of the score matching loss (see Eq. (A4)) for the coordinates and cell, respectively, and the atom type loss (compare with D3PM objective in Eq. (A13)):

$$L = \lambda_{\text{coord}} L_{\text{coord}} + \lambda_{\text{cell}} L_{\text{cell}} + \lambda_{\text{types}} L_{\text{types}}, \quad (\text{A40})$$

where

$$L_{\text{coord}} = \sum_{t=1}^T \sigma_t(n)^2 \mathbb{E}_{q(\mathbf{x}_0)} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [\|\mathbf{s}_{\mathbf{x},\theta}(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0)\|_2^2], \quad (\text{A41})$$

$$L_{\text{cell}} = \sum_{t=1}^T (1 - \bar{\alpha}_t) \sigma_t(n)^2 \mathbb{E}_{q(\mathbf{L}_0)} \mathbb{E}_{q(\mathbf{L}_t|\mathbf{L}_0)} [\|\mathbf{s}_{\mathbf{L},\theta}(\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) - \nabla_{\mathbf{L}_t} \log q(\mathbf{L}_t|\mathbf{L}_0)\|_2^2] \quad (\text{A42})$$

$$L_{\text{types}} = \mathbb{E}_{q(\mathbf{a}_0)} \left[\sum_{t=2}^T \mathbb{E}_{q(\mathbf{a}_t|\mathbf{a}_0)} [D_{\text{KL}} [q(\mathbf{a}_{t-1}|\mathbf{a}_t, \mathbf{a}_0) \parallel p_{\theta}(\mathbf{a}_{t-1}|\mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t)]] \right]$$

$$- \lambda_{\text{CE}} \log p_{\boldsymbol{\theta}}(\mathbf{a}_0 | \mathbf{X}_t, \mathbf{L}_t, \mathbf{A}_t, t) - \mathbb{E}_{q(\mathbf{a}_1 | \mathbf{a}_0)} [\log p_{\boldsymbol{\theta}}(\mathbf{a}_0 | \mathbf{X}_1, \mathbf{L}_1, \mathbf{A}_1, 1)] \Bigg]. \quad (\text{A43})$$

For simplicity, Eqs. (A41) and (A43) show the loss only for a single atom's coordinates and specie, respectively; the overall losses for coordinates and atom types sum over all atoms in a structure.

B Fine-tuning the score network for property-guided generation

Here we discuss our fine-tuning procedure of the score model to enable *property-guided* generation via classifier-free guidance [45].

B.1 Fine-tuning the score network with adapter modules

Leveraging the large-scale unlabeled Alex-MP-20 dataset enables MatterGen to generate a broad distribution of stable material structures via reverse diffusion, driven by unconditional scores. To facilitate conditional generation with classifier-free guidance, the property-conditioned scores need to be learned through a labeled dataset. However, labeled datasets, often limited in size and diversity, present challenges in learning the conditional scores from scratch.

To enable rapid learning of the conditional scores in the sparsely-labeled data regime, we propose to fine-tune the unconditional score network with additional trainable adapter modules. Each adapter layer is a combination of an MLP layer and a zero-initialized mix-in layer [44], so MatterGen still outputs the learned unconditional scores at initialization. This is desired because the unconditional scores have been optimized to generate stable materials during pre-training, which is a prerequisite for modeling the property-conditional distribution of materials.

The additional adapter modules consist of an embedding layer f_{embed} for the property label that outputs a property embedding $\mathbf{g}$, and a series of adapter layers, one before each message-passing layer (four in total). The adapter layer augments the atom embedding of the original GemNet score network to incorporate property information. Concretely, at the L -th interaction layer, given the property embedding $\mathbf{g}$ and the intermediate node hidden representation $\{\mathbf{H}_j^{(L)}\}_{j=1}^n$, the property-augmented node hidden representation $\{\mathbf{H}_j^{\prime(L)}\}_{j=1}^n$ is given by:

$$\mathbf{H}_j^{\prime(L)} = \mathbf{H}_j^{(L)} + f_{\text{mixin}}^{(L)} \left(f_{\text{adapter}}^{(L)}(\mathbf{g}) \right) \cdot \mathbb{I}(\text{property is not null}), \quad (\text{B44})$$

where $f_{\text{mixin}}^{(L)}$ is the L -th mix-in layer, which is a zero-initialized linear layer without bias weights. $f_{\text{adapter}}^{(L)}$ is the L -th adapter layer, which is a two-layer MLP model. The indicator function $\mathbb{I}(\text{property is not null})$ ensures the model outputs the unconditional score when no conditional label is given. The adapter modules add additional weights of f_{embed} , $f_{\text{adapter}}^{(L)}$, and $f_{\text{mixin}}^{(L)}$ to each layer of the model.

The fine-tuning process uses the same training objective as the unconditional pre-training stage with conditional property labels incorporated. When fine-tuning finishes, the score network is able to predict both conditional and unconditional scores. The fine-tuned model enables us to generate structures satisfying the property condition without a major sacrifice in terms of stability and novelty. With the unconditional score network as a strong initialization, the fine-tuning procedure is more computation- and sample-efficient than re-training from scratch if the labeled dataset is only sparsely labeled.

B.2 Classifier-free guidance

To generate samples conditioned on a specific value c of a property, we adopt classifier-free diffusion guidance [45] throughout this work. In classifier-free guidance, a guidance factor γ is applied to the conditional distribution $p(\mathbf{M}_t|c)$, such that

$$\begin{aligned} p_\gamma(\mathbf{M}_t|c) &\propto p(c|\mathbf{M}_t)^\gamma p(\mathbf{M}_t) \\ &\propto \left(\frac{p(\mathbf{M}_t|c)}{p(\mathbf{M}_t)} \right)^\gamma p(\mathbf{M}_t) \\ &\propto p(\mathbf{M}_t|c)^\gamma p(\mathbf{M}_t)^{1-\gamma} \end{aligned} \tag{B45}$$

is used instead of $p(\mathbf{M}_t)$ when evaluating the model score during the reverse process in the conditional setting. We adopt a value of $\gamma = 2$ in all conditional generation experiments.

B.2.1 Continuous case

The conditional score follows from Eq. (B45) by taking gradients of the logarithm w.r.t. continuous variables in $\mathbf{M}_t$. For example, for fractional coordinates $\mathbf{X}_t$ we have

$$\nabla_{\mathbf{X}_t} \ln p_\gamma(\mathbf{X}_t|c) = \gamma \nabla_{\mathbf{X}_t} \ln q(\mathbf{X}_t|c) + (1 - \gamma) \nabla_{\mathbf{X}_t} \ln q(\mathbf{X}_t). \tag{B46}$$

Practically, learning a conditional score $\nabla_{\mathbf{X}_t} \ln q(\mathbf{X}_t|c)$ equates to concatenating a latent embedding $\mathbf{g}_c \in \mathbb{R}^d$ of the condition c to the score model $\mathbf{s}_\theta(\mathbf{M}_t, \mathbf{g}_c, t)$ during score matching. The unconditional score $\nabla_{\mathbf{X}_t} \ln p(\mathbf{X}_t)$ is obtained by providing a null embedding for the condition, i.e., using $\mathbf{s}_\theta(\mathbf{M}_t, \mathbf{g}_c = \text{null}, t)$. When we condition on multiple properties, the conditional score for N properties with embeddings $\mathbf{g}_{c_i}$ is obtained by $\mathbf{s}_\theta(\mathbf{M}_t, \mathbf{g}_{c_1}, \mathbf{g}_{c_2}, \dots, \mathbf{g}_{c_N}, t)$.

B.2.2 Discrete case

The model’s task in denoising discrete atom types $\mathbf{a}$ is to fit and predict $\tilde{q}(\mathbf{a}_{t-1}|\mathbf{a}_t, c)$. Following Eq. (4) in [65], we can rewrite this as

$$\tilde{q}(\mathbf{a}_{t-1}|\mathbf{a}_t, c) \propto \sum_{\mathbf{a}_0} q(\mathbf{a}_{t-1}, \mathbf{a}_t|\mathbf{a}_0) \cdot \tilde{q}(\mathbf{a}_0|\mathbf{a}_t, c).$$

Thus, the predictive task is to approximate $p_\theta(\mathbf{a}_0|\mathbf{a}_t, c) \approx \tilde{q}(\mathbf{a}_0|\mathbf{a}_t, c)$. For this distribution we can perform classifier-free guidance as follows:

$$\begin{aligned} \tilde{q}_\gamma(\mathbf{a}_0|\mathbf{a}_t, c) &\propto \tilde{q}(c|\mathbf{a}_0, \mathbf{a}_t)^\gamma \cdot \tilde{q}(\mathbf{a}_0|\mathbf{a}_t) \\ &= \left(\frac{\tilde{q}(\mathbf{a}_0|c, \mathbf{a}_t) \cdot \tilde{q}(c|\mathbf{a}_t)}{\tilde{q}(\mathbf{a}_0|\mathbf{a}_t)} \right)^\gamma \cdot \tilde{q}(\mathbf{a}_0|\mathbf{a}_t) \\ &\propto \tilde{q}(\mathbf{a}_0|c, \mathbf{a}_t)^\gamma \cdot \tilde{q}(\mathbf{a}_0|\mathbf{a}_t)^{1-\gamma}. \end{aligned}$$